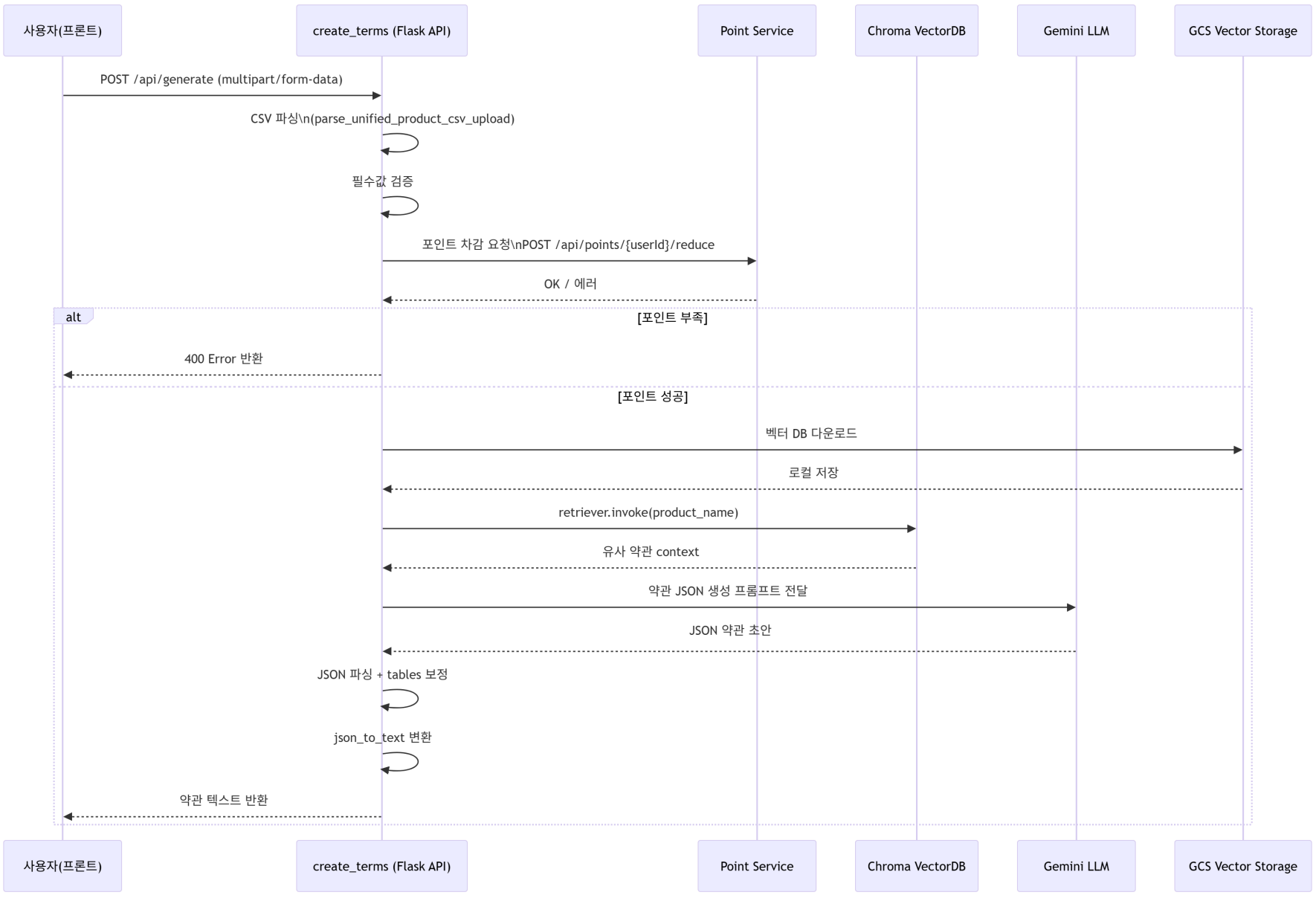
Project Portfolio

1. AI 기반 약관 통합 관리 플랫폼

1) ‘약관 이해도 평가 제도’의 평가 기준을 활용한 약관 초안 품질 이슈 해결

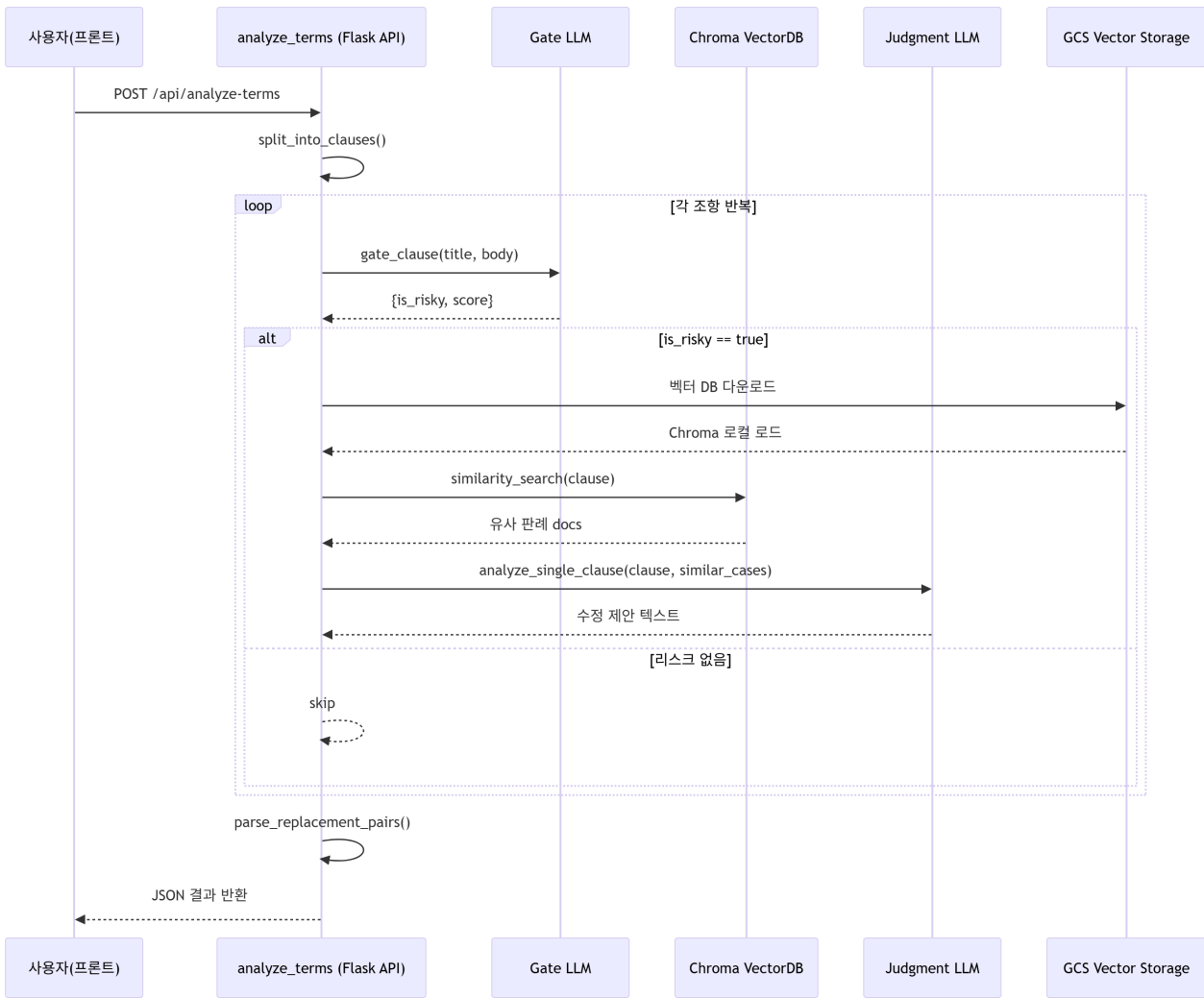
- 약관 초안 생성 시퀀스 다이어그램



- 문제 원인
  - 단순 프롬프트 기반 생성 결과와 비교했을 때 품질이 개선되었음을 객관적으로 증명할 근거가 부족
  - 생성 결과는 형식적으로는 약관 구조를 따르고 있었으나 조항 수와 관 구성의 일관성 및 가독성이 부족
- 해결 과정
  - 금융위원회 ‘약관 이해도 평가제도’의 평가 항목과 기준을 프롬프트 제약 조건으로 변환
  - 우수·양호 등급 약관이 왜 높은 점수를 받았는지 평가 기준 기반으로 구조적으로 분석하여 공통 패턴 도출
  - 테스트
    - 평가 기준 적용한 자체 평가 모델 설계하여 단순 LLM 프롬프트로 작성한 초안과 품질 비교
- 결과
  - 구조적 패턴 반영으로 조항 수 및 관 구성의 일관성 확보 및 문서 안정성 향상과 약관 가독성, 시인성 개선
  - 내부 평가 모델 기준 평균 15점 이상 상승

2) 법제처 판례 선별 및 2차 게이트 구조로 리스크 분석 품질 향상

- 약관 분석 시퀀스 다이어그램

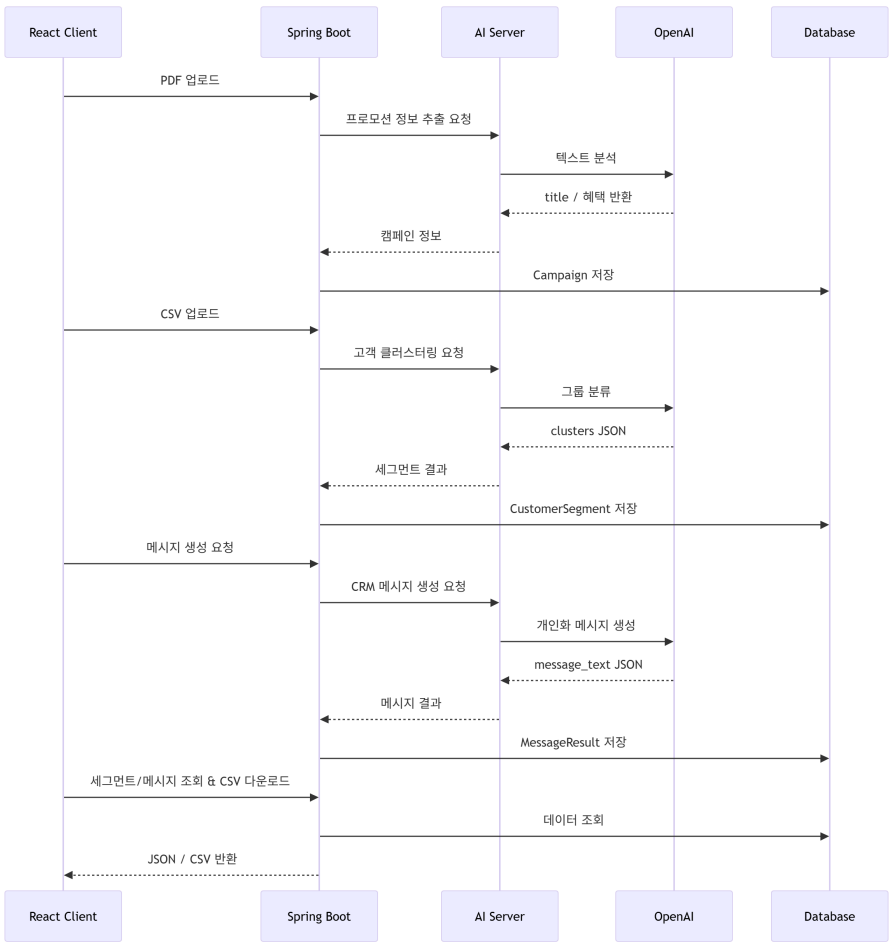


- 문제 원인
  - 수집한 판례 전체를 벡터DB에 저장할 경우 “약관”이 단순 언급된 경우와 “약관의 효력·해석이 쟁점인 사건”이 구분되지 않음
  - 모든 조항에 판례 벡터DB에 대한 RAG 검색이 수행되어 불필요한 검색 호출 반복으로 토큰 사용량 증가, 정상 조항까지 리스크로 분류되는 문제 발생
- 해결 과정
  - 법제처 API로 판례 수집 후 LLM을 활용하여 ‘약관 조항의 해석·효력이 판결의 핵심 쟁점’인 판례만 저장
  - 리스크 분석시 1차 게이트 모델 설계 근거 문장(evidence) 필수 포함, 리스크 가능성 점수화 (0~100)
- 결과
  - 수정 제안의 법적 타당성 강화 및 유사 판례 매칭 신뢰도 상승
  - 실제 리스크 조항 중심 분석으로 분석 비용 및 토큰 사용량 감소

2. AI 기반 개인화 자동 메시지 생성기

서비스 구조 재설계 및 AI 세그먼트 추론 고도화

- 시퀀스 다이어그램



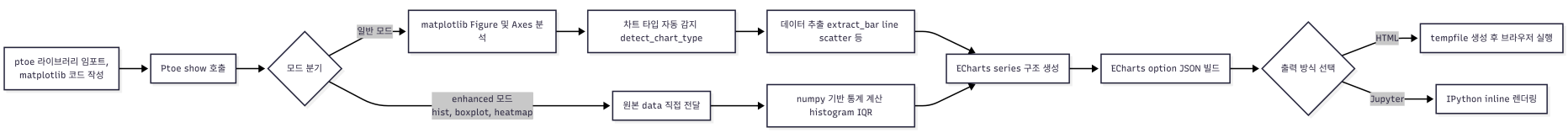
- 문제 원인 (초기버전)
  - 프로모션·고객 정보를 텍스트로 직접 입력받고 컬럼명만 모델에 전달하는 비현실적 구조, 생성 메시지를 실제 고객 데이터에 매핑할 수 없는 설계
  - 소비자 특성 논문을 기반으로 벡터DB를 구축했으나 통신사 프로모션 도메인과 직접적 연관성이 낮아 실효성 부족
  - 내부 툴 특성을 고려하지 않은 WebClient 기반 비동기 구조로, 실제 병목(AI 호출)과 무관한 과설계 상태
  - 클릭률 등 성과 데이터를 활용한 성장 구조를 설계했으나, 지속적 데이터 축적을 위해서는 별도의 프로모션 관리 플랫폼 수준의 확장이 필요한 한계 존재
- 해결 과정
  - 전체 아키텍처 재설계
    - 프로모션 정보는 PDF 업로드 기반 파싱, 고객 데이터가 담긴 CSV 파일 업로드하여 프로모션 정보 + 고객 데이터 기반 타겟 그룹 매핑
    - 타겟 그룹 매핑 결과와 메시지 생성 결과를 포함한 CSV 다운로드 기능 구현
  - 통신사 프로모션과 직접적인 연관성이 부족한 논문 DB RAG 기능 제거
  - Spring 최신 동기 HTTP 클라이언트인 RestClient로 전환하여 불필요한 비동기 구조 제거하고 WebClient와 유사한 DSL 유지하여 코드 가독성 및 디버깅 용이성 향상

- 결과
  - 실제 업무 흐름과 유사한 마케팅 자동화 구조 완성
  - PDF/CSV 기반 입력 및 결과 매핑 CSV 다운로드 기능으로 즉시 발송 플랫폼 연계 가능
  - 불필요한 RAG 제거로 시스템 단순화 및 고객 데이터 기반 세그먼트 정확도 향상
  - RestClient로의 전환으로 코드 복잡도 감소 및 유지보수성 향상

3. ptoe (Matplotlib → ECharts 변환 라이브러리)

기존 Matplotlib 문법은 그대로 유지하면서 출력만 ECharts 기반 인터랙티브 차트로 매핑하도록 설계

- 프로젝트 아키텍처



- 문제 원인
  - Matplotlib은 Python에서 가장 널리 쓰이는 데이터 시각화 라이브러리이지만 정적 이미지 기반 출력과 제한적인 디자인 표현
  - ECharts는 시각적으로 세련되고 인터랙티브한 웹 기반 차트를 제공하지만 JavaScript 기반으로 동작하여 기존 matplotlib 사용자에게는 학습 장벽이 존재
  - hist, heatmap, boxplot과 같이 통계 계산이 필요한 차트는 단순 결과만 보여주는 것으로는 정확히 구현 불가

- 해결 과정
  - 주요 차트 유형을 추상화하고, Axes 내부 state를 분석하여 x/y 데이터, label, stacked 여부, step 옵션, 축 타입(category/value)을 자동 판별
  - 추출한 시각화 속성을 기반으로 ECharts option JSON 구조로 변환
  - 변환된 ECharts 옵션을 기반으로 HTML 템플릿 자동 생성하여 브라우저에서 바로 실행 가능한 결과물 출력
  - show() 호출 시 원본 데이터를 한 번 더 전달받도록 설계하여 내부에서 통계값 계산 후 ECharts 형식으로 변환

- 결과
  - Python 분석 환경에서 작성한 코드 그대로 유지하여 추가 JavaScript 작성 없이 인터랙티브 차트 생성 가능
  - 웹 대시보드 환경에 즉시 적용 가능
  - 차트 타입 자동 인식 및 옵션 매핑 자동화